Loan Default Prediction

Comparing Classic ML and Neural Networks under Class Imbalance

Minwoo Yoo · Nathaniel Badalov

Home Credit Default Risk · Kaggle

Problem & Motivation

The task

Predict the probability that a loan applicant defaults (binary classification, $\texttt{TARGET} \in \{0, 1\}$).

Why this matters

- Lenders need accurate risk scoring to balance approval volume and credit losses.
- False negatives cost the full unpaid principal — far more than the foregone interest of a false positive.

The challenge: class imbalance

- Only $\sim$**8.07%** of applicants default — the data is severely imbalanced.
- A naive "always predict no default" baseline scores 92% accuracy with **zero recall**.

Dataset · Home Credit Default Risk

Size

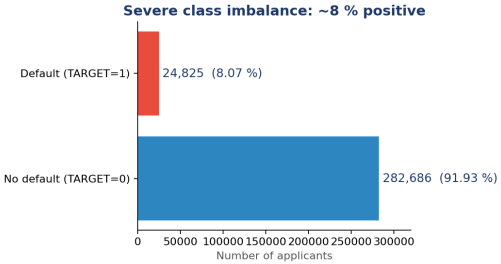
- Train: **307,511** applicants × 122 features
- Test: 48,744 applicants (Kaggle hidden labels)

Class imbalance

- Default rate ≈ **8.07%** (24,825 / 307,511)

Strongest signals

- `EXT_SOURCE_1/2/3` external credit scores ($|\rho| \approx 0.16$–0.18)



Shared Pipeline

Preprocessing — *identical for all five models*

- **80/20 stratified** train–val split (`random_state = 42`)
- Drop `SK_ID_CURR`; keep IDs only for the submission file
- Mean imputation for numeric NaN; one-hot encode 16 categoricals → **244 features**
- `StandardScaler` fitted on train only (essential for LR / MLP)

Tuning & validation

- `GridSearchCV` with `PredefinedSplit` (single train→val fold)
- Scoring: `roc_auc`; best parameter setting picked per model
- CV CSVs saved to `result/home_credit/cv_results/GridSearchCV/`

Classic ML Models

Logistic Regression — val AUC **0.7487**

`C` $\in \{0.01, 0.1, 1, 10\}$, `tol` $\in \{10^{-5}, 10^{-4}, 10^{-3}\}$
`class_weight=balanced`

Random Forest — val AUC **0.7470**

`n_estimators` $\in \{100, 200\}$
`min_samples_split` $\in \{2, 20, 100\}$
`min_samples_leaf` $\in \{1, 20, 100\}$,
`class_weight=balanced`

HistGradientBoosting — val AUC **0.7595** $\star$

`learning_rate` $\in \{0.01, 0.05, 0.1\}$
`max_iter` $\in \{100, 200\}$
`min_samples_leaf` $\in \{20, 100\}$

Findings

- **HistGradientBoosting wins** — boosted trees usually dominate tabular data.
- **Random Forest overfits**: train AUC 0.96 vs val 0.747; regularized leaf=20+ wins.
- Logistic Regression is surprisingly close — most signal is linear in `EXT_SOURCE_*`.

Neural Network Models

Shallow MLP val AUC **0.7405**

hidden $(64,)$ $\sim$15.7K parameters

Deep MLP val AUC **0.7416**

hidden $(128, 64, 32)$ $\sim$42K parameters

Common setup

- ReLU activation · Adam optimizer · `early_stopping = True`

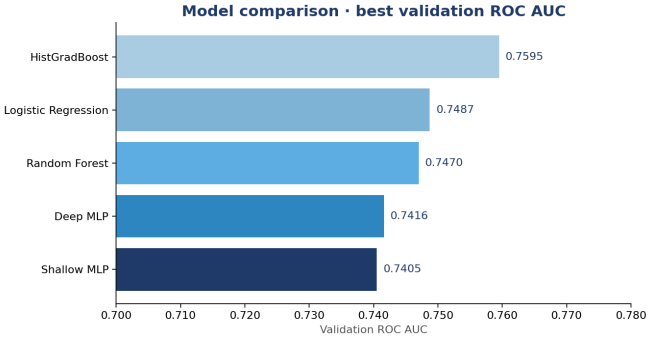
Hyperparameter grid

- `alpha` (L2 reg.) $\in \{10^{-4}, 10^{-3}\}$
- `learning_rate_init` $\in \{0.001, 0.005\}$
- Best for both: $\alpha = 10^{-3}$, $\texttt{lr} = 0.005$

Findings

- **Depth doesn't help**
  0.7405 vs 0.7416 — a 0.001 difference, well within noise.
- Tripling the parameter count bought *nothing*.
- Both NNs land slightly *below* the classic shallow learners.
- Bottleneck is the *feature set*, not the model class.

Combined Leaderboard

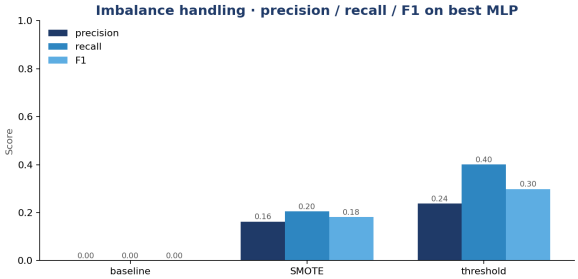


Model	Val AUC
HistGradBoost	**0.7595**
Logistic Reg.	0.7487
Random Forest	0.7470
Deep MLP	0.7416
Shallow MLP	0.7405

Tree boosting wins by ∼**1 AUC point** — consistent with the literature on tabular data.

Imbalance Handling on the Best MLP

Method	Threshold	ROC AUC	PR AUC	Precision	Recall	F1
Baseline	0.500	0.745	0.222	0.00	0.00	0.00
SMOTE	0.500	**0.648**	0.134	0.16	0.21	0.18
Threshold tuning ⋆	0.164	0.745	0.222	0.24	**0.40**	**0.30**



Headline

- Baseline: AUC 0.745 but recall **0%** — no prediction crosses 0.5.
- SMOTE **hurts**: AUC dropped 0.10.
- Threshold tuning at 0.164 lifts recall to **40%**.

Conclusions

What we learned

- **HistGradientBoosting** wins overall at val ROC AUC $= 0.7595$.
- **Neural networks** tie LR and RF — depth alone does not unlock new performance on tabular data.
- **Threshold tuning** on the MLP lifts recall from 0% to 40%; SMOTE hurt AUC.

Limitations

- Only main `application_*.csv` tables — auxiliary tables typically add several AUC points.
- `PredefinedSplit` is fast but optimistic; full k-fold CV would tighten confidence intervals.

Future work

Thank You

Questions?

Recording «INSERT RECORDING LINK BEFORE SUBMISSION»

Code `github.com/ymw0414/loan-default-imbalance-classification`

Team Minwoo Yoo · Nathaniel Badalov